

The increase in potential due to the LINK operation, therefore, is at most  $\alpha(n)$ . The amortized cost of the LINK operation is  $O(1) + \alpha(n) = O(\alpha(n))$ . ■

**Lemma 19.13**

The amortized cost of each FIND-SET operation is  $O(\alpha(n))$ .

**Proof** Suppose that the  $q$ th operation is a FIND-SET and that the find path contains  $s$  nodes. The actual cost of the FIND-SET operation is  $O(s)$ . We will show that no node's potential increases due to the FIND-SET and that at least  $\max\{0, s - (\alpha(n) + 2)\}$  nodes on the find path have their potential decrease by at least 1.

We first show that no node's potential increases. Lemma 19.10 takes care of all nodes other than the root. If  $x$  is the root, then its potential is  $\alpha(n) \cdot x.rank$ , which does not change due to the FIND-SET operation.

Now we show that at least  $\max\{0, s - (\alpha(n) + 2)\}$  nodes have their potential decrease by at least 1. Let  $x$  be a node on the find path such that  $x.rank > 0$  and  $x$  is followed somewhere on the find path by another node  $y$  that is not a root, where  $\text{level}(y) = \text{level}(x)$  just before the FIND-SET operation. (Node  $y$  need not *immediately* follow  $x$  on the find path.) All but at most  $\alpha(n) + 2$  nodes on the find path satisfy these constraints on  $x$ . Those that do not satisfy them are the first node on the find path (if it has rank 0), the last node on the path (i.e., the root), and the last node  $w$  on the path for which  $\text{level}(w) = k$ , for each  $k = 0, 1, 2, \dots, \alpha(n) - 1$ .

Consider such a node  $x$ . It has positive rank and is followed somewhere on the find path by nonroot node  $y$  such that  $\text{level}(y) = \text{level}(x)$  before the path compression occurs. We claim that the path compression decreases  $x$ 's potential by at least 1. To prove this claim, let  $k = \text{level}(x) = \text{level}(y)$  and  $i = \text{iter}(x)$  before the path compression occurs. Just prior to the path compression caused by the FIND-SET, we have

$$\begin{aligned} x.p.rank &\geq A_k^{(i)}(x.rank) && \text{(by the definition (19.5) of } \text{iter}(x) \text{),} \\ y.p.rank &\geq A_k(y.rank) && \text{(by the definition (19.3) of } \text{level}(y) \text{),} \\ y.rank &\geq x.p.rank && \text{(by Corollary 19.5 and because} \\ &&& \text{ } y \text{ follows } x \text{ on the find path).} \end{aligned}$$

Putting these inequalities together gives

$$\begin{aligned} y.p.rank &\geq A_k(y.rank) \\ &\geq A_k(x.p.rank) && \text{(because } A_k(j) \text{ is strictly increasing)} \\ &\geq A_k(A_k^{(i)}(x.rank)) \\ &= A_k^{(i+1)}(x.rank) && \text{(by the definition (3.30) of functional iteration).} \end{aligned}$$

Because path compression makes  $x$  and  $y$  have the same parent, after path compression we have  $x.p.rank = y.p.rank$ . The parent of  $y$  might change due to the path compression, but if it does, the rank of  $y$ 's new parent compared with the rank of  $y$ 's parent before path compression is either the same or greater. Since  $x.rank$  does not change,  $x.p.rank = y.p.rank \geq A_k^{(i+1)}(x.rank)$  after path compression. By the definition (19.5) of the iter function, the value of  $\text{iter}(x)$  increases from  $i$  to at least  $i + 1$ . By Lemma 19.10,  $\phi_q(x) \leq \phi_{q-1}(x) - 1$ , so that  $x$ 's potential decreases by at least 1.

The amortized cost of the FIND-SET operation is the actual cost plus the change in potential. The actual cost is  $O(s)$ , and we have shown that the total potential decreases by at least  $\max\{0, s - (\alpha(n) + 2)\}$ . The amortized cost, therefore, is at most  $O(s) - (s - (\alpha(n) + 2)) = O(s) - s + O(\alpha(n)) = O(\alpha(n))$ , since we can scale up the units of potential to dominate the constant hidden in  $O(s)$ . (See Exercise 19.4-6.) ■

Putting the preceding lemmas together yields the following theorem.

**Theorem 19.14**

A sequence of  $m$  MAKE-SET, UNION, and FIND-SET operations,  $n$  of which are MAKE-SET operations, can be performed on a disjoint-set forest with union by rank and path compression in  $O(m \alpha(n))$  time.

**Proof** Immediate from Lemmas 19.7, 19.11, 19.12, and 19.13. ■

**Exercises**

**19.4-1**

Prove Lemma 19.4.

**19.4-2**

Prove that every node has rank at most  $\lfloor \lg n \rfloor$ .

**19.4-3**

In light of Exercise 19.4-2, how many bits are necessary to store  $x.rank$  for each node  $x$ ?

**19.4-4**

Using Exercise 19.4-2, give a simple proof that operations on a disjoint-set forest with union by rank but without path compression run in  $O(m \lg n)$  time.

**19.4-5**

Professor Dante reasons that because node ranks increase strictly along a simple path to the root, node levels must monotonically increase along the path. In other words, if  $x.rank > 0$  and  $x.p$  is not a root, then  $level(x) \leq level(x.p)$ . Is the professor correct?

**19.4-6**

The proof of Lemma 19.13 ends with scaling the units of potential to dominate the constant hidden in the  $O(s)$  term. To be more precise in the proof, you need to change the definition (19.7) of the potential function to multiply each of the two cases by a constant, say  $c$ , that dominates the constant in the  $O(s)$  term. How must the rest of the analysis change to accommodate this updated potential function?

**★ 19.4-7**

Consider the function  $\alpha'(n) = \min \{k : A_k(1) \geq \lg(n + 1)\}$ . Show that  $\alpha'(n) \leq 3$  for all practical values of  $n$  and, using Exercise 19.4-2, show how to modify the potential-function argument to prove that performing a sequence of  $m$  MAKE-SET, UNION, and FIND-SET operations,  $n$  of which are MAKE-SET operations, on a disjoint-set forest with union by rank and path compression takes  $O(m\alpha'(n))$  time.

---

**Problems**
**19-1 Offline minimum**

In the *offline minimum problem*, you maintain a dynamic set  $T$  of elements from the domain  $\{1, 2, \dots, n\}$  under the operations INSERT and EXTRACT-MIN. The input is a sequence  $S$  of  $n$  INSERT and  $m$  EXTRACT-MIN calls, where each key in  $\{1, 2, \dots, n\}$  is inserted exactly once. Your goal is to determine which key is returned by each EXTRACT-MIN call. Specifically, you must fill in an array *extracted*[1 :  $m$ ], where for  $i = 1, 2, \dots, m$ , *extracted*[ $i$ ] is the key returned by the  $i$ th EXTRACT-MIN call. The problem is “offline” in the sense that you are allowed to process the entire sequence  $S$  before determining any of the returned keys.

- a.* Consider the following instance of the offline minimum problem, in which each operation INSERT( $i$ ) is represented by the value of  $i$  and each EXTRACT-MIN is represented by the letter E:

4, 8, E, 3, E, 9, 2, 6, E, E, E, 1, 7, E, 5 .

Fill in the correct values in the *extracted* array.

To develop an algorithm for this problem, break the sequence  $S$  into homogeneous subsequences. That is, represent  $S$  by

$I_1, E, I_2, E, I_3, \dots, I_m, E, I_{m+1}$ ,

where each  $E$  represents a single EXTRACT-MIN call and each  $I_j$  represents a (possibly empty) sequence of INSERT calls. For each subsequence  $I_j$ , initially place the keys inserted by these operations into a set  $K_j$ , which is empty if  $I_j$  is empty. Then execute the OFFLINE-MINIMUM procedure.

OFFLINE-MINIMUM( $m, n$ )

```

1  for  $i = 1$  to  $n$ 
2      determine  $j$  such that  $i \in K_j$ 
3      if  $j \neq m + 1$ 
4           $extracted[j] = i$ 
5          let  $l$  be the smallest value greater than  $j$  for which set  $K_l$  exists
6           $K_l = K_j \cup K_l$ , destroying  $K_j$ 
7  return  $extracted$ 
```

- b.* Argue that the array *extracted* returned by OFFLINE-MINIMUM is correct.
- c.* Describe how to implement OFFLINE-MINIMUM efficiently with a disjoint-set data structure. Give as tight a bound as you can on the worst-case running time of your implementation.

### 19-2 Depth determination

In the [depth-determination problem](#), you maintain a forest  $\mathcal{F} = \{T_i\}$  of rooted trees under three operations:

MAKE-TREE( $v$ ) creates a tree whose only node is  $v$ .

FIND-DEPTH( $v$ ) returns the depth of node  $v$  within its tree.

GRAFT( $r, v$ ) makes node  $r$ , which is assumed to be the root of a tree, become the child of node  $v$ , which is assumed to be in a different tree from  $r$  but may or may not itself be a root.

- a.* Suppose that you use a tree representation similar to a disjoint-set forest:  $v.p$  is the parent of node  $v$ , except that  $v.p = v$  if  $v$  is a root. Suppose further that you implement GRAFT( $r, v$ ) by setting  $r.p = v$  and FIND-DEPTH( $v$ ) by following the find path from  $v$  up to the root, returning a count of all nodes other than  $v$  encountered. Show that the worst-case running time of a sequence of  $m$  MAKE-TREE, FIND-DEPTH, and GRAFT operations is  $\Theta(m^2)$ .

By using the union-by-rank and path-compression heuristics, you can reduce the worst-case running time. Use the disjoint-set forest  $\mathcal{S} = \{S_i\}$ , where each set  $S_i$  (which is itself a tree) corresponds to a tree  $T_i$  in the forest  $\mathcal{F}$ . The tree structure within a set  $S_i$ , however, does not necessarily correspond to that of  $T_i$ . In fact, the implementation of  $S_i$  does not record the exact parent-child relationships but nevertheless allows you to determine any node's depth in  $T_i$ .

The key idea is to maintain in each node  $v$  a “pseudodistance”  $v.d$ , which is defined so that the sum of the pseudodistances along the simple path from  $v$  to the root of its set  $S_i$  equals the depth of  $v$  in  $T_i$ . That is, if the simple path from  $v$  to its root in  $S_i$  is  $v_0, v_1, \dots, v_k$ , where  $v_0 = v$  and  $v_k$  is  $S_i$ 's root, then the depth of  $v$  in  $T_i$  is  $\sum_{j=0}^k v_j.d$ .

- b.* Give an implementation of MAKE-TREE.
- c.* Show how to modify FIND-SET to implement FIND-DEPTH. Your implementation should perform path compression, and its running time should be linear in the length of the find path. Make sure that your implementation updates pseudodistances correctly.
- d.* Show how to implement GRAFT( $r, v$ ), which combines the sets containing  $r$  and  $v$ , by modifying the UNION and LINK procedures. Make sure that your implementation updates pseudodistances correctly. Note that the root of a set  $S_i$  is not necessarily the root of the corresponding tree  $T_i$ .
- e.* Give a tight bound on the worst-case running time of a sequence of  $m$  MAKE-TREE, FIND-DEPTH, and GRAFT operations,  $n$  of which are MAKE-TREE operations.

### 19-3 Tarjan's offline lowest-common-ancestors algorithm

The **lowest common ancestor** of two nodes  $u$  and  $v$  in a rooted tree  $T$  is the node  $w$  that is an ancestor of both  $u$  and  $v$  and that has the greatest depth in  $T$ . In the **offline lowest-common-ancestors problem**, you are given a rooted tree  $T$  and an arbitrary set  $P = \{\{u, v\}\}$  of unordered pairs of nodes in  $T$ , and you wish to determine the lowest common ancestor of each pair in  $P$ .

To solve the offline lowest-common-ancestors problem, the LCA procedure on the following page performs a tree walk of  $T$  with the initial call  $\text{LCA}(T.\text{root})$ . Assume that each node is colored WHITE prior to the walk.

- a.* Argue that line 10 executes exactly once for each pair  $\{u, v\} \in P$ .
- b.* Argue that at the time of the call  $\text{LCA}(u)$ , the number of sets in the disjoint-set data structure equals the depth of  $u$  in  $T$ .

```

LCA( $u$ )
1  MAKE-SET( $u$ )
2  FIND-SET( $u$ ).ancestor =  $u$ 
3  for each child  $v$  of  $u$  in  $T$ 
4      LCA( $v$ )
5      UNION( $u$ ,  $v$ )
6      FIND-SET( $u$ ).ancestor =  $u$ 
7   $u$ .color = BLACK
8  for each node  $v$  such that  $\{u, v\} \in P$ 
9      if  $v$ .color == BLACK
10         print “The lowest common ancestor of”
             $u$  “and”  $v$  “is” FIND-SET( $v$ ).ancestor

```

- c.* Prove that LCA correctly prints the lowest common ancestor of  $u$  and  $v$  for each pair  $\{u, v\} \in P$ .
- d.* Analyze the running time of LCA, assuming that you use the implementation of the disjoint-set data structure in Section 19.3.

---

## Chapter notes

Many of the important results for disjoint-set data structures are due at least in part to R. E. Tarjan. Using aggregate analysis, Tarjan [427, 429] gave the first tight upper bound in terms of the very slowly growing inverse  $\hat{\alpha}(m, n)$  of Ackermann’s function. (The function  $A_k(j)$  given in Section 19.4 is similar to Ackermann’s function, and the function  $\alpha(n)$  is similar to  $\hat{\alpha}(m, n)$ . Both  $\alpha(n)$  and  $\hat{\alpha}(m, n)$  are at most 4 for all conceivable values of  $m$  and  $n$ .) An upper bound of  $O(m \lg^* n)$  was proven earlier by Hopcroft and Ullman [5, 227]. The treatment in Section 19.4 is adapted from a later analysis by Tarjan [431], which is based on an analysis by Kozen [270]. Harfst and Reingold [209] give a potential-based version of Tarjan’s earlier bound.

Tarjan and van Leeuwen [432] discuss variants on the path-compression heuristic, including “one-pass methods,” which sometimes offer better constant factors in their performance than do two-pass methods. As with Tarjan’s earlier analyses of the basic path-compression heuristic, the analyses by Tarjan and van Leeuwen are aggregate. Harfst and Reingold [209] later showed how to make a small change to the potential function to adapt their path-compression analysis to these one-pass variants. Goel et al. [182] prove that linking disjoint-set trees randomly yields the

same asymptotic running time as union by rank. Gabow and Tarjan [166] show that in certain applications, the disjoint-set operations can be made to run in  $O(m)$  time.

Tarjan [428] showed that a lower bound of  $\Omega(m \hat{\alpha}(m, n))$  time is required for operations on any disjoint-set data structure satisfying certain technical conditions. This lower bound was later generalized by Fredman and Saks [155], who showed that in the worst case,  $\Omega(m \hat{\alpha}(m, n)) (\lg n)$ -bit words of memory must be accessed.



*Part VI   Graph Algorithms*

---

## Introduction

Graph problems pervade computer science, and algorithms for working with them are fundamental to the field. Hundreds of interesting computational problems are couched in terms of graphs. This part touches on a few of the more significant ones.

Chapter 20 shows how to represent a graph in a computer and then discusses algorithms based on searching a graph using either breadth-first search or depth-first search. The chapter gives two applications of depth-first search: topologically sorting a directed acyclic graph and decomposing a directed graph into its strongly connected components.

Chapter 21 describes how to compute a minimum-weight spanning tree of a graph: the least-weight way of connecting all of the vertices together when each edge has an associated weight. The algorithms for computing minimum spanning trees serve as good examples of greedy algorithms (see Chapter 15).

Chapters 22 and 23 consider how to compute shortest paths between vertices when each edge has an associated length or “weight.” Chapter 22 shows how to find shortest paths from a given source vertex to all other vertices, and Chapter 23 examines methods to compute shortest paths between every pair of vertices.

Chapter 24 shows how to compute a maximum flow of material in a flow network, which is a directed graph having a specified source vertex of material, a specified sink vertex, and specified capacities for the amount of material that can traverse each directed edge. This general problem arises in many forms, and a good algorithm for computing maximum flows can help solve a variety of related problems efficiently.

Finally, Chapter 25 explores matchings in bipartite graphs: methods for pairing up vertices that are partitioned into two sets by selecting edges that go between the sets. Bipartite-matching problems model several situations that arise in the real world. The chapter examines how to find a matching of maximum cardinality; the

“stable-marriage problem,” which has the highly practical application of matching medical residents to hospitals; and assignment problems, which maximize the total weight of a bipartite matching.

When we characterize the running time of a graph algorithm on a given graph  $G = (V, E)$ , we usually measure the size of the input in terms of the number of vertices  $|V|$  and the number of edges  $|E|$  of the graph. That is, we denote the size of the input with two parameters, not just one. We adopt a common notational convention for these parameters. Inside asymptotic notation (such as  $O$ -notation or  $\Theta$ -notation), and *only* inside such notation, the symbol  $V$  denotes  $|V|$  and the symbol  $E$  denotes  $|E|$ . For example, we might say, “the algorithm runs in  $O(VE)$  time,” meaning that the algorithm runs in  $O(|V||E|)$  time. This convention makes the running-time formulas easier to read, without risk of ambiguity.

Another convention we adopt appears in pseudocode. We denote the vertex set of a graph  $G$  by  $G.V$  and its edge set by  $G.E$ . That is, the pseudocode views vertex and edge sets as attributes of a graph.